

자바 유틸리티와 예외처리 정리

오늘의 강의 학습 요약

금일 강의는 자바에서 자주 활용되는 유틸리티 API를 빠르게 정리한 뒤, 예외 처리의 개념과 구현 방식(try-catch, throws 위임, 명시적 throw, 사용자 정의 예외, 다중 catch, 예외 계층, 예외 정보 조회, finally, 오버라이딩 시 throws 규칙)까지 실습 중심으로 확장하는 흐름으로 구성되었습니다.

특히 인터페이스의 핵심 목적(메서드 강제, 느슨한 결합)과 중첩/익명 클래스가 람다로 이어지는 관계를 상기한 다음, 날짜 API의 진화(Date → Calendar → LocalDate)와 문자열 처리(StringTokenizer), 래퍼 클래스(파싱, 범위 상수, 오토박싱/언박싱)까지 “왜 이런 API/문법이 필요한지”를 맥락과 함께 강조했습니다.

1. 인터페이스, 중첩, 익명, 람다 핵심

인터페이스는 메서드 구현 강제와 느슨한 결합을 위해 필수이며, 중첩 클래스는 특정 외부 클래스에서만 쓰이는 내부 구현을 숨기기 위해 활용되고, 익명 클래스는 람다의 기반 문법으로 연결됩니다.

추상 클래스는 사용 빈도가 상대적으로 낮아 “왜 쓰는가” 정도의 목적을 이해하는 수준이 중요합니다.

인터페이스는 프로젝트에서 매우 자주 쓰이며, 핵심 용도는 구현 클래스에 특정 메서드를 강제하는 것과 느슨한 결합(Loose Coupling)을 만드는 것입니다.

- 인터페이스는 구현체를 바꿔도 호출부 코드를 최소로 변경하게 하여 유지보수성을 높입니다.
- 인터페이스는 하위 클래스에 필요한 동작을 강제하여 일관된 설계를 가능하게 합니다.

중첩 클래스(이너 클래스)는 “특정 클래스만 내부 클래스를 사용”하는 구조에서 파일 분리로 인한 복잡성을 줄이기 위해 사용됩니다.

GUI에서 화면 생성 코드와 이벤트 처리 코드를 분리할 수 있는데, 이벤트 처리 클래스가 특정 화면 클래스에서만 쓰인다면 중첩 클래스로 넣어 은닉성과 관리성을 높일 수 있습니다.

컴파일 결과 중첩 클래스는 별도 `.class` 파일로 만들어지며, 파일명에 `$` 가 포함되는 것이 특징입니다.

익명 클래스는 실무에서도 자주 등장하며, 람다 표현식은 익명 클래스를 더 간결하게 표현한 형태로 이해할 수 있습니다.

- 람다는 “메서드 형태”에 집중한 표현식이므로, 파라미터/리턴 유무 조합의 제한된 형태로 정리하는 것이 효과적입니다.

핵심 키워드

#인터페이스

#느슨한결합

#중첩클래스

#익명클래스

#람다

2. String, StringBuilder, Date/Calendar 개요

String은 불변(immutable)이며 리터럴/생성자 생성 방식의 차이를 이해해야 하고, **StringBuilder**는 잦은 변경에 유리하며, **Date**는 구식 API로 대체 흐름을 알고 **Calendar**의 생성 방식(싱글톤 패턴)을 이해합니다.

String은 한 번 생성되면 값이 바뀌지 않는 불변 객체입니다. 이 특성은 안전성과 공유에 유리하지만, 문자열 변경이 잦으면 성능이 불리할 수 있습니다.

String은 리터럴로 생성할 수도 있고 `new` 로 생성할 수도 있으며, 두 방식은 객체 관리(특히 동일 리터럴 공유 여부) 관점에서 차이가 발생할 수 있습니다.

String의 주요 메서드는 “무엇이 가능했는지”를 기능 중심으로 기억하고, 상세 시그니처는 필요할 때 API 문서로 확인하는 방식이 실전적입니다.

기능 범주	예시(의미)
길이/조회	문자열 길이 확인, 특정 문자 얻기.
검색	특정 문자의 위치 얻기.
변환	대소문자 변환.
추출	부분 문자열(부분열) 얻기.

StringBuilder는 내부 버퍼를 사용하여 문자열이 커졌다/줄어드는 변경 작업에 효율적입니다. 최종적으로는 **String**으로 변환하여 사용하는 경우가 많습니다.

StringBuilder를 **String**으로 바꿀 때는 `toString()` 을 사용하며, 이는 `Object.toString()` 이 주소 형태를 주는 것과 달리 실제 문자열 값을 반환하도록 재정의되어 있다는 점이 중요합니다.

Date는 오래된 API로 대부분 deprecated 되었으며, “현재 시간 찍기” 같은 단발성 목적에만 제한적으로 이해해 두는 정도로 정리했습니다.

Calendar는 `new` 대신 `Calendar.getInstance()` 로 생성하며, 이는 싱글톤 패턴 관점(인스턴스를 하나만 관리)과 연결해 설명되었습니다.

핵심 키워드

#String불변

#StringBuilder

#toString

#Date

#Calendar

3. 싱글톤 패턴 구현과 Calendar API 이해

싱글톤은 인스턴스를 하나만 만들도록 생성자를 `private`으로 막고, `static` 메서드로 단일 인스턴스를 반환하며, `Calendar`는 추상 클래스, `protected` 생성자 때문에 `getInstance()` 방식만 허용됩니다.

싱글톤 디자인 패턴은 “단 하나의 인스턴스만 존재해야 하는 대상”을 만들 때 사용합니다. 핵심은 외부에서 `new`로 생성하지 못하게 막고, 클래스 내부에서 한 번만 생성해 재사용하도록 만드는 것입니다.

싱글톤 구현의 필수 단계는 다음 구조로 정리됩니다.

- 생성자를 `private`으로 선언하여 외부에서 인스턴스 생성을 차단합니다.
- 클래스 내부에 `private static` 필드로 단일 인스턴스를 보관합니다.
- `public static getInstance()` 메서드에서 인스턴스가 `null`이면 생성하고, 아니면 기존 인스턴스를 반환합니다.
- 외부 호출은 클래스명 `.getInstance()` 형태로만 이루어지게 됩니다.

```
public class Singleton {
    private static Singleton instance;

    private Singleton() {}

    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}
```

`Calendar` API를 문서로 확인했을 때 `java.util.Calendar`는 `public abstract class`이므로 `new`가 불가능하며, 생성자는 `protected`라서 패키지가 다르고 상속 관계가 아니면 호출할 수 없습니다. 이 구조 때문에 `getInstance()`를 통해 인스턴스를 얻는 방식이 사실상 강제됩니다.

`Calendar` 내부에는 중첩 클래스(예: `Builder`)가 존재하며, `public static final int` 형태의 상수 필드를 다수 제공하여 `get(int field)` 호출 시 “어떤 값”을 얻을지 지정하는 구조입니다.

핵심 키워드 `#싱글톤` `#private`생성자 `#getInstance` `#Calendar`추상클래스
`#protected`생성자

4. Calendar로 현재 날짜, 특정 날짜 다루기

Calendar는 `get(상수필드)`로 연, 월, 일, 시, 분, 초를 얻고, 월은 0부터 시작하므로 +1 보정이 필요하며, `set()`으로 특정 날짜를 설정할 수 있습니다.

Calendar에서 값을 얻을 때는 `get(int field)`를 사용하며, `field`에는 `Calendar.YEAR` 같은 상수 값을 넣습니다. 이 상수는 전부 `int`이며 API가 제공하는 “필드 선택자”입니다.

12시간제/24시간제는 `HOUR` 와 `HOUR_OF_DAY` 차이로 구분됩니다.

Calendar의 월(month)은 0부터 11까지로 관리되므로, 출력 시 사람이 보는 월로 맞추려면 반드시 +1 보정이 필요합니다. 또한 문자열 결합 중 +1 이 붙는 문제를 피하려면 괄호로 연산 우선순위를 명확히 해야 합니다.

특정 날짜를 설정할 때는 `set()` 계열 메서드로 연/월/일 또는 연/월/일/시/분 등을 지정할 수 있습니다.

목적	대표 메서드/포인트
현재 날짜/시간 얻기	<code>Calendar.getInstance()</code> 후 <code>get()</code> 호출로 각 필드 조회.
필드 지정 조회	<code>get(Calendar.YEAR)</code> 처럼 상수로 조회 대상을 지정.
월 보정	<code>Calendar.MONTH</code> 는 0 기반이므로 출력 시 +1 처리.
특정 날짜 설정	<code>set(...)</code> 으로 원하는 날짜/시간을 주입.

핵심 키워드 `#Calendar.get` `#Calendar상수필드` `#MONTH0기반` `#HOUR_OF_DAY` `#set`

5. LocalDate와 DateTimeFormatter 기반 파싱

`LocalDate`는 `java.time` 패키지의 최신 날짜 API로 생성자가 없고 `now/of/parse`로 생성하며, `plus/minus`로 날짜 연산을 안전하게 처리하고, 포맷이 다른면 `DateTimeFormatter` 패턴으로 `parse`를 보조합니다.

`LocalDate`는 `java.time` 패키지에 있으며, 생성자가 없어 `new` 로 만들지 않습니다. 대신 정적 메서드로 인스턴스를 얻습니다.

`LocalDate`가 제공하는 핵심 기능은 “날짜 계산을 API가 책임지고 처리”해준다는 점입니다. 과거에는 윤년 등 복잡한 계산을 직접 해야 했지만, `LocalDate`는 `plusXxx`, `minusXxx` 로 안전하게 처리합니다.

`LocalDate` 생성/연산의 대표 형태는 다음과 같습니다.

- `LocalDate.now()` 는 현재 날짜를 반환합니다.
- `LocalDate.of(y, m, d)` 는 특정 날짜를 생성합니다.
- `date.minusDays(n)`, `date.minusMonths(n)`, `date.minusYears(n)` 는 과거 날짜로 이동합니다.
- `date.plusDays(n)` 등은 미래 날짜로 이동합니다.

문자열을 날짜로 바꾸는 `parse` 는 입력 문자열 형식이 ISO-8601(예: `2029-12-24`)이면 포맷터 없이도 가능하지만, `2029/12/24` 또는 `2029년 12월 24일` 처럼 형식이 다른면 `DateTimeFormatter` 로 패턴을 명시해야 합니다.

패턴 심볼	의미	주의점
y	연도(Year)	yyyy 형태가 일반적입니다.
M	월(Month)	대문자 M 이 월이며 소문자 m 은 분입니다.
d	일(Day of month)	대문자 D 는 Day of year(연중 일수)입니다.
H / h	시(Hour)	H 는 24시간, h 는 12시간입니다.

```
String s = "2029/12/24";
LocalDate d = LocalDate.parse(s, DateTimeFormatter.ofPattern("yyyy/MM/dd"));
```

`LocalDate`는 날짜만 다루고, 시간까지 함께 다루려면 `LocalDateTime` 을 사용하며 사용 방식은 유사하다는 점도 함께 정리되었습니다.

핵심 키워드

- #LocalDate
- #java.time
- #plusMinus
- #parse
- #DateTimeFormatter

6. StringTokenizer로 구분자 문자열 분리

StringTokenizer는 문자열 split과 유사하게 구분자로 토큰을 분리하며, 기본 구분자는 공백이고, `hasMoreTokens/nextToken` 반복으로 안전하게 순회하며, 여러 구분자를 한 번에 지정할 수 있습니다.

StringTokenizer는 `java.util`에 있으며, 구분자로 문자열을 분리해 토큰(token) 단위로 순회하는 데 특화된 클래스입니다.

기본 구분자는 공백이므로 공백 분리라면 구분자를 생략할 수 있고, 공백 이외 구분자를 쓰려면 생성자에 명시해야 합니다.

토큰 개수를 모르는 상황에서 `nextToken()` 만 반복 호출하면 예외가 날 수 있으므로, 일반적으로 `hasMoreTokens()` 를 조건으로 반복하는 형태를 사용합니다.

메서드	의미
<code>hasMoreTokens()</code>	다음 토큰이 존재하면 <code>true</code> 를 반환합니다.
<code>nextToken()</code>	다음 토큰 문자열을 반환하고 내부 포인터를 이동합니다.
<code>countTokens()</code>	남아 있는 토큰 개수를 반환합니다.
여러 구분자 지정	구분자 문자열에 <code>/,:</code> 처럼 나열하면 모두 구분자로 처리됩니다.

핵심 키워드

`#StringTokenizer`

`#토큰`

`#구분자`

`#hasMoreTokens`

`#nextToken`

7. 래퍼 클래스: 파싱, 상수, 오토박싱

래퍼 클래스는 기본형 8개를 객체로 감싼 클래스이며, `parseInt` 같은 파싱 메서드와 `MIN_VALUE`/`MAX_VALUE` 상수를 제공하고, 오토박싱/언박싱으로 기본형과 참조형이 자연스럽게 호환됩니다.

래퍼 클래스는 기본형(primitive) 8개를 각각 대응하는 참조형 클래스로 제공하는 개념입니다. 대표적으로 `int` 는 `Integer` 이며, `char` 는 `Character` 처럼 이름이 다른 것도 있습니다.

래퍼 클래스의 목적은 "기본형에 대한 유틸 기능(메서드/상수)을 클래스 형태로 제공"하는 데 있습니다. 가장 대표적인 활용은 문자열 숫자를 실제 숫자로 바꾸는 파싱입니다.

목적	대표 예시
문자열 → 정수	<code>Integer.parseInt("10")</code>
문자열 → 실수	<code>Float.parseFloat("3.14")</code>
타입 범위 상수	<code>Integer.MIN_VALUE</code> , <code>Integer.MAX_VALUE</code>
비교/선택 유틸	<code>Integer.max(a,b)</code> , <code>Integer.min(a,b)</code>

오토박싱/언박싱은 기본형과 래퍼 타입이 자연스럽게 오갈 수 있게 지원하는 기능입니다. 이 덕분에 `Object[]` 같은 "객체만 담는 구조"에 기본형 값을 넣어도 내부적으로 박싱이 일어나 동작할 수 있습니다.

- 기본형 → 래퍼로 감싸지는 동작을 오토박싱이라고 합니다.
- 래퍼 → 기본형으로 풀리는 동작을 오토 언박싱이라고 합니다.

또한 래퍼 클래스 생성자들은 deprecated인 경우가 많아, `new Integer(...)` 같은 방식은 지양하고 정적 팩토리/파싱 메서드 위주로 사용하는 관점이 강조되었습니다.

핵심 키워드

#WrapperClass

#Integer.parseInt

#MIN_VALUE

#오토박싱

#언박싱

8. enum으로 상수 집합 관리와 switch 활용

enum은 다수의 상수를 타입 안전하게 묶어 관리하며, enum명.상수로 사용하고, switch의 분기 값으로도 사용 가능하며, name/ordinal/values로 메타 정보를 얻을 수 있습니다.

상수가 많아지면 `public static final` 필드가 다수 생겨 코드가 비효율적이고 관리가 어려워질 수 있습니다. enum은 “정해진 값의 집합”을 타입으로 묶어, 존재하지 않는 값 사용을 컴파일 단계에서 차단합니다.

enum 사용 방식은 `EnumType.SUNDAY` 처럼 `enum명.상수` 형태입니다. 또한 switch 문에서 enum 값을 분기 값으로 사용할 수 있습니다.

enum에서 자주 쓰는 메서드는 다음과 같습니다.

- `name()` 은 상수의 이름(String)을 반환합니다.
- `ordinal()` 은 선언 순서(0부터 시작)를 반환합니다.
- `values()` 는 모든 상수의 배열을 반환합니다.

핵심 키워드

`#enum`

`#switch`

`#name`

`#ordinal`

`#values`

9. 예외와 예외처리 개념: 정상/비정상 종료

예외는 실행 중 발생하는 문제로 비정상 종료를 유발하며, 예외처리는 예외를 “수정”하는 것이 아니라 사용자에게 원인을 알리고 프로그램이 정상 종료되도록 흐름을 제어하는 것입니다.

자바에서 컴파일 오류(문법 오류로 인한 컴파일 에러)와 실행 중 발생하는 예외(Exception)는 구분됩니다. 예외는 실행 중 발생하며, 처리하지 않으면 프로그램이 중간에 멈추는 비정상 종료가 됩니다.

예외는 “발생 자체를 0으로 만들 수 없다”는 관점이 중요합니다. 핵심은 예외가 발생했을 때 사용자에게 원인을 알려주고, 비정상 종료가 아닌 정상 종료로 마무리되도록 사후 처리를 하는 것입니다.

예외처리는 문제가 발생한 코드를 다시 되돌려 수정해 재실행하는 의미가 아니라, 발생 이후의 프로그램 흐름을 제어해 나머지 코드가 실행되도록 만드는 의미입니다.

구분	의미
정상 종료	main 메서드의 끝까지 실행이 도달하는 상태입니다.
비정상 종료	예외로 인해 실행이 중단되어 main 끝까지 가지 못하는 상태입니다.
예외처리 목표	사용자에게 원인 안내 및 프로그램의 정상 종료 보장입니다.
오해 포인트	예외처리는 발생한 문제 코드를 “수정”하는 행위가 아닙니다.

핵심 키워드

[#예외](#)[#비정상종료](#)[#정상종료](#)[#런타임예외](#)[#사후처리](#)

10. try-catch 동작 원리, 다중 catch, 다형성

try에는 실행 코드를, catch에는 예외 처리 코드를 두며, 예외 발생 시 JVM이 적합한 예외 객체를 만들어 던지고 catch가 잡아 처리하며, 다중 catch는 다양한 예외를 분기 처리하고, catch는 하위 타입부터 작성해야 합니다.

예외가 발생한 곳에서 처리하는 기본 문법은 `try { ... } catch(예외타입 e) { ... }` 입니다. try 블록은 실행할 코드이고, catch 블록은 try 내부에서 예외가 발생했을 때만 실행됩니다.

예외 처리의 내부 동작을 관점적으로 정리하면, 실행 중 예외 발생 시 JVM이 적합한 예외 클래스를 찾아 객체를 만들고(내부), 이를 던진 다음(throw) catch가 이를 받아 예외 정보를 활용하여 처리합니다.

catch의 예외 타입은 "발생 예외를 처리할 수 있는 타입"이어야 하며, 전혀 관련 없는 타입을 적으면 잡지 못해 그대로 비정상 종료됩니다.

다중 catch는 try 블록 내에서 서로 다른 예외가 발생할 수 있을 때 catch를 여러 개 나열하여 각각 처리하는 방식입니다. 이때 계층 구조상 하위(더 구체적인) 예외부터 위(더 포괄적인) 예외 순으로 작성해야 합니다.

- catch는 구체적인 예외 타입부터 작성해야 하며, 상위를 먼저 쓰면 아래 catch가 도달 불가로 컴파일 오류가 납니다.
- 마지막 catch로 `Exception` 을 두는 것은 "예상치 못한 예외"까지 잡기 위한 관례적 안전장치가 됩니다.

catch에는 다형성이 적용되어, 예를 들어 `ArithmeticException` 은 `RuntimeException` 이나 `Exception` 으로도 잡을 수 있습니다. 다만 실무에서는 원인 파악을 위해 구체적으로 잡는 것을 권장합니다.

핵심 키워드

#try-catch

#다중catch

#예외계층

#다형성

#도달불가catch

11. 예외 정보 조회: getMessage, printStackTrace

예외 객체는 메시지와 발생 위치 정보를 가지며, `getMessage`는 문자열을 반환해 출력이 필요하고, `printStackTrace`는 콘솔에 스택 추적을 직접 출력해 디버깅에 유용합니다.

예외 객체에는 발생 원인에 대한 정보가 담겨 있으며, 이를 확인하는 대표 메서드가 `getMessage()` 와 `printStackTrace()` 입니다.

`getMessage()` 는 문자열을 반환하므로 `System.out.println(e.getMessage())` 처럼 출력과 함께 써야 합니다. 반면 `printStackTrace()` 는 자체적으로 콘솔 출력 기능이 있어 호출만으로 스택 정보를 보여줍니다.

`printStackTrace()` 는 예외가 “어느 메서드 체인에서, 어느 라인에서” 발생했는지 추적하는 데 유용하여 디버깅 목적에 자주 사용됩니다.

핵심 키워드

#getMessage

#printStackTrace

#스택트레이스

#디버깅

#예외정보

12. try-finally와 finally 의미

`try-finally`는 `catch` 없이도 문법상 가능하지만 예외 처리는 되지 않으며, `finally`는 예외 여부와 무관하게 반드시 실행되어야 하는 코드를 배치하는 용도로 사용됩니다.

`catch`가 없는 `try-finally` 는 문법적으로는 가능하지만, 예외를 잡아 정상 종료로 만드는 예외처리 목적에는 부합하지 않습니다.

`finally`는 예외 발생 여부와 관계없이 실행되어야 하는 코드를 배치하는 용도로 이해하는 것이 핵심입니다. 실무에서는 `try-catch-finally` 형태가 일반적이며, `finally`를 통해 “반드시 실행할 작업”을 보장합니다.

핵심 키워드

#finally

#try-finally

#반드시실행

#예외처리구조

#흐름제어

13. throws로 예외 위임(집중화 처리)

`throws`는 예외를 호출자에게 위임하여 예외 처리 위치를 메인 등 한 곳으로 집중화할 수 있으며, 메인 이외 메서드는 `throws`로 넘기고 최종 호출부에서 `try-catch`로 처리하는 패턴이 실무적으로 유리합니다.

예외를 위임하는 방식은 “발생된 곳에서 `try-catch`로 처리하지 않고 호출한 곳으로 넘기는 것”입니다. 문법은 메서드 시그니처 뒤에 `throws` 예외 타입 을 선언합니다.

메서드 호출 관계가 `main → A() → B()` 일 때, B에서 예외가 발생하면 B가 `throws` 로 A에게 넘기고, A가 다시 `throws` 로 main에게 넘긴 뒤 main에서 최종적으로 `try-catch` 로 처리할 수 있습니다.

이 방식의 장점은 예외 처리 위치를 한 곳으로 모아(집중화) 유지보수와 디버깅이 쉬워진다는 점입니다. 여러 클래스/메서드에 산발적으로 `try-catch`가 퍼지면 “어디서 처리했는지” 추적이 어려워집니다.

또한 `throws`는 하나만 던지는 것이 아니라, 발생 가능한 예외가 여러 개라면 `throws` 에 여러 타입을 나열할 수 있으며, 최종 처리부(main)는 다중 `catch`로 각각 처리할 수 있습니다.

핵심 키워드

`#throws`

`#위임처리`

`#집중화`

`#메서드호출체인`

`#다중throws`

14. throw로 명시적 예외 발생과 사용자 정의 예외

`throw`는 개발자가 조건에 따라 예외를 직접 발생시키는 키워드이며, 시스템이 예외로 판단하지 않는 업무 규칙 위반(예: 잔액 부족)을 예외로 다루기 위해 필요하고, 사용자 정의 예외는 `Exception`을 상속해 메시지 생성자를 제공하여 구현합니다.

명시적 예외 발생은 시스템이 자동으로 예외를 던지지 않는 상황을 개발자가 “예외로 간주”하여 강제로 발생시키는 방식입니다. 예를 들어 출금 로직에서 잔액 부족은 문법/시스템 관점의 예외가 아니지만, 업무 규칙 관점에서는 예외 상황이 될 수 있습니다.

명시적 예외 발생 문법은 `throw new` 예외 클래스 (“메시지”) 입니다. 또한 `throws` (위임)와 `throw` (발생)는 키워드 목적이 완전히 다르므로 반드시 구분해야 합니다.

- `throw` 는 예외를 “발생”시키는 키워드입니다.
- `throws` 는 예외를 “위임”하는 선언입니다.

잔액 부족처럼 API에 적절한 예외 클래스가 없는 경우, 사용자 정의 예외 클래스를 만들어 의미를 명확히 할 수 있습니다. 사용자 정의 예외는 `Exception` 을 상속하고, 메시지를 받는 생성자에서 `super(message)` 를 호출하는 형태가 핵심입니다.

```
public class InsufficientBalanceException extends Exception {  
    public InsufficientBalanceException(String message) {  
        super(message);  
    }  
}
```

이후 업무 규칙 위반 지점에서 다음처럼 발생시키고, 상위로 위임하거나 최종 처리부에서 잡아 정상 종료 흐름을 보장합니다.

```
throw new InsufficientBalanceException("잔액 부족");
```

핵심 키워드

#`throw`

#사용자정의예외

#`Exception`상속

#업무규칙예외

#`super`메시지

15. 오버라이딩(상속)에서 throws 규칙

메서드 오버라이딩 시 자식 메서드의 throws는 부모가 던진 예외와 같거나 하위 타입만 선언 가능하며, 부모에 throws가 없으면 자식도 새로운 체크 예외를 추가할 수 없습니다.

부모 메서드에 throws 가 선언되어 있을 때, 자식이 오버라이딩하면서 던지는 예외는 “부모가 던진 예외와 같거나 계층적으로 더 하위(더 구체적) 예외”만 허용됩니다.

부모 메서드에 throws가 없다면, 자식이 오버라이딩하면서 갑자기 throws를 추가하는 것은 예외 범위를 확대하는 것이므로 허용되지 않는다는 점이 실습으로 확인되었습니다.

또한 부모가 특정 계열(예: RuntimeException)을 던진다고 했을 때, 자식은 같은 계열의 하위 예외는 던질 수 있지만 전혀 관계없는 계열(예: IOException)로 바꾸는 것은 허용되지 않습니다.

상황	자식 오버라이딩 시 throws
부모 throws 없음	자식이 체크 예외를 새로 추가할 수 없습니다.
부모가 RuntimeException throws	자식은 같은 타입 또는 하위(Runtime 계열 하위)만 가능합니다.
부모가 특정 예외 throws	자식은 같은 타입 또는 하위 타입만 가능합니다.
무관한 예외로 변경	계층 관계가 없으면 불가능합니다.

핵심 키워드

#오버라이딩

#throws 규칙

#예외계층

#상위하위제약

#체크예외